Ecole Normale Supérieure de l'Enseignement Technique Mohammedia
Université Hassan II de Casablanca

ENSET

المدرسة العليا لأساتذة التعليم التقني المحمدية
جامعة الحسن الثاني بالدار البيضاء

Département Mathématiques et Informatique

Rapport de Travaux Dirigés

Filière : **BDCC – Semestre 5**

Architecture microservices d'un système E-Bank

Partie I – Conception, implémentation et validation

EBANK

ARCHITECTURE MICROSERVICES – SYSTÈMES DISTRIBUÉS

Microservices et systèmes distribués

Java 21 – Spring Boot – Spring Cloud

Réalisé par :
Abdelali Saadali

Encadré par :
Pr. Mohamed YOUSSFI

Réalisation : octobre 2026

Année universitaire : 2026–2027

Avant-propos

Le présent document constitue le livrable académique de la première partie du projet pratique « E-Bank » réalisé dans le cadre du module *Microservices et systèmes distribués* à l'École Normale Supérieure de l'Enseignement Technique de Mohammedia. Il expose l'approche suivie, les choix de conception, l'organisation des services, les étapes de développement et les résultats des essais réalisés.

Le travail repose sur une démarche incrémentale : chaque couche a été construite puis vérifiée avant l'introduction des mécanismes distribués. Les illustrations proviennent des captures réalisées pendant le développement. La documentation distingue les éléments effectivement implémentés des améliorations envisageables ; les exemples de données sont signalés comme tels lorsque cela est nécessaire.

Remerciements

Je tiens à exprimer ma reconnaissance au professeur Mohamed YOUSSFI pour le cadre pédagogique du module et pour la démarche pratique favorisant la compréhension des services distribués. Je remercie également le Département Mathématiques et Informatique de l'ENSET Mohammedia pour les enseignements techniques mobilisés dans ce travail. La réalisation progressive du projet m'a permis d'approfondir Spring Boot, la persistance JPA, les architectures REST et les mécanismes de communication et de tolérance aux pannes.

Résumé

Ce rapport présente la conception et la réalisation d'une application bancaire pédagogique reposant sur une architecture de microservices. Deux services métier indépendants assurent respectivement la gestion des clients et celle des comptes bancaires. Ils utilisent Java 21, Spring Boot 3.5.10, Spring Data JPA et deux bases H2 distinctes en mémoire. Le projet intègre ensuite un registre de découverte Netflix Eureka, une passerelle HTTP Spring Cloud Gateway, un client déclaratif OpenFeign et un mécanisme de résilience fondé sur Spring Cloud Circuit Breaker et Resilience4j.

La réalisation se distingue par la séparation des responsabilités métier, par l'usage de noms logiques de services et par l'enrichissement dynamique d'un compte bancaire à partir d'informations distantes. Les vérifications combinent compilation Maven, tests automatisés ciblés, consultation de bases H2, requêtes Postman, observation du tableau de bord Eureka et expérimentation du mode dégradé. La fonctionnalité de repli permet de continuer à consulter les données locales d'un compte lorsque le service client est indisponible. Les résultats sont documentés par un inventaire de captures et une traçabilité Git.

Mots-clés : microservices, systèmes distribués, Spring Boot, Spring Cloud, REST, Eureka, Gateway, OpenFeign, Resilience4j, JPA, H2, Java, Maven.

Index des captures d'écran

Les captures sont identifiées par des codes stables **S01**–**S25** issus de l'archive d'origine. Elles sont réorganisées ci-dessous selon le parcours de réalisation plutôt que selon l'ordre arbitraire de l'archive. Le numéro de figure et la page renvoient à leur insertion dans le présent rapport.

Code	Contenu de la preuve	Renvoi
S09	Démarrage initial du service client et traces Spring Boot	fig. 3.1, p. 9
S23	Vérification HTTP du point d'entrée de contrôle /hello	fig. 3.2, p. 9
S20	Exploration de la base H2 des clients à la création	fig. 3.3, p. 9
S02	Compilation et vérifications du service client dans VS Code	fig. 3.4, p. 10
S19	Création d'un client par requête HTTP POST	fig. 3.7, p. 11
S15	Consultation des clients par requête HTTP GET directe	fig. 3.8, p. 12
S04	Inspection des enregistrements de la table CUSTOMER	fig. 3.9, p. 12
S22	Exécution des tests du composant métier client	fig. 3.5, p. 10
S05	Implémentation et lancement des composants du service client	fig. 3.6, p. 11
S01	Code du contrôleur bancaire et vérification de son exécution	fig. 3.10, p. 14
S10	Compilation et exécution du service bancaire	fig. 3.11, p. 14
S14	Inspection initiale de la structure H2 bancaire	fig. 3.12, p. 15
S24	Lecture de la table BANK_ACCOUNT dans H2	fig. 3.13, p. 15
S07	Création d'un compte bancaire par requête POST directe	fig. 3.14, p. 16
S03	Consultation d'un compte bancaire par requête GET directe	fig. 3.15, p. 16
S13	Console Eureka au démarrage, sans instance inscrite	fig. 4.1, p. 19
S25	Enregistrement de CUSTOMER-SERVICE dans Eureka	fig. 4.2, p. 19

Code	Contenu de la preuve	Renvoi
S08	Enregistrement simultané de CUSTOMER-SERVICE et EBANK-SERVICE	fig. 4.3, p. 20
S21	Enregistrement des trois composants applicatifs, incluant Gateway	fig. 4.4, p. 21
S06	Consultation de /customers à travers le port 9999	fig. 4.5, p. 22
S11	Consultation de /accounts à travers le port 9999	fig. 4.6, p. 22
S18	Création d'un client à travers la passerelle HTTP	fig. 4.7, p. 23
S16	Création d'un compte bancaire à travers la passerelle HTTP	fig. 4.8, p. 23
S12	Consultation enrichie d'un compte et de son client par OpenFeign	fig. 5.2, p. 27

Une vingt-cinquième image de l'archive (S17) représente une interface extérieure de gestion de clé API. Elle est volontairement omise : elle ne constitue pas une preuve du projet E-Bank et pourrait divulguer des informations sensibles. Les 24 captures pertinentes sont intégrées.

Table des matières

Avant-propos i

Remerciements ii

Résumé iii

Index des captures d'écran iv

Table des figures x

Liste des tableaux xi

Liste des abréviations xii

Introduction générale 1

1 Contexte et fondements du projet 2

1.1 Introduction . . . 2
1.2 Problématique et motivation . . . 2
1.3 Objectifs fonctionnels et techniques . . . 2
1.4 Microservices et indépendance des données . . . 3
1.5 Méthodologie de développement . . . 3
1.6 Conclusion . . . 3

2 Conception du système distribué 4

2.1 Vue d'ensemble . . . 4
2.2 Rôle et adressage des composants . . . 5
2.3 Architecture en couches des services métier . . . 5
2.4 Modèle de données . . . 5

2.4.1 Client . . . 5
2.4.2 Compte bancaire . . . 5
2.5 Conception des échanges HTTP . . . 6
2.6 Limites du modèle de démonstration . . . 6
2.7 Conclusion . . . 6

3 Réalisation des services métier 7
3.1 Environnement de développement . . . 7
3.2 Implémentation de Customer-Service . . . 7
3.2.1 Initialisation et persistance . . . 7
3.2.2 Service métier et contrôleur HTTP . . . 8
3.2.3 Premières validations . . . 8
3.2.4 Tests automatisés du service client . . . 8
3.3 Implémentation de EBank-Service . . . 13
3.3.1 Entité et repository . . . 13
3.3.2 Règles métier . . . 13
3.3.3 Publication de l'API . . . 13
3.3.4 Validation en environnement local . . . 13
3.4 Conclusion . . . 17

4 Découverte de services et passerelle HTTP 18
4.1 Netflix Eureka : registre de découverte . . . 18
4.2 Enregistrement des applications métier . . . 18
4.3 Spring Cloud Gateway . . . 20
4.4 Routage de couche applicative et répartition . . . 20
4.5 Validation de la passerelle . . . 21
4.6 Précision sur les avertissements Eureka . . . 21
4.7 Conclusion . . . 21

5 Communication interservices et tolérance aux pannes 25
5.1 Problème de composition des données . . . 25
5.2 Intégration de Spring Cloud OpenFeign . . . 25
5.3 Enrichissement d'une entité par un DTO distant . . . 26
5.4 Déroulement d'une requête interservices . . . 26
5.5 Validation fonctionnelle d'OpenFeign . . . 26

5.6 Circuit Breaker et mode dégradé 27
5.7 Expérimentation de l'indisponibilité 28
5.8 Limites de la stratégie de résilience 28
5.9 Traçabilité et progression des travaux 28
5.10 Bilan des validations 28
5.11 Conclusion 30

Conclusion générale **31**

Bibliographie et ressources **32**

A Éléments complémentaires **33**
A.1 Structure synthétique des projets Maven 33
A.2 Exemples de requêtes pour la reproduction 33
A.3 Ordre de lancement recommandé 34
A.4 Note de gestion des preuves visuelles 34

Table des figures

2.1 Architecture fonctionnelle de la partie I : services métier, découverte, passerelle et données isolées 4

3.1 Démarrage initial du service client et traces Spring Boot 9
3.2 Vérification HTTP du point d'entrée de contrôle /hello 9
3.3 Exploration de la base H2 des clients à la création 9
3.4 Compilation et vérifications du service client dans VS Code 10
3.5 Exécution des tests du composant métier client 10
3.6 Implémentation et lancement des composants du service client 11
3.7 Création d'un client par requête HTTP POST 11
3.8 Consultation des clients par requête HTTP GET directe 12
3.9 Inspection des enregistrements de la table CUSTOMER 12
3.10 Code du contrôleur bancaire et vérification de son exécution 14
3.11 Compilation et exécution du service bancaire 14
3.12 Inspection initiale de la structure H2 bancaire 15
3.13 Lecture de la table BANK_ACCOUNT dans H2 15
3.14 Création d'un compte bancaire par requête POST directe 16
3.15 Consultation d'un compte bancaire par requête GET directe 16

4.1 Console Eureka au démarrage, sans instance inscrite 19
4.2 Enregistrement de CUSTOMER-SERVICE dans Eureka 19
4.3 Enregistrement simultané de CUSTOMER-SERVICE et EBANK-SERVICE 20
4.4 Enregistrement des trois composants applicatifs, incluant Gateway 21
4.5 Consultation de /customers à travers le port 9999 22
4.6 Consultation de /accounts à travers le port 9999 22
4.7 Création d'un client à travers la passerelle HTTP 23

4.8 Création d'un compte bancaire à travers la passerelle HTTP 23

5.1 Séquence simplifiée de récupération enrichie d'un compte bancaire 26
5.2 Consultation enrichie d'un compte et de son client par OpenFeign 27

Liste des tableaux

1.1 Correspondance entre objectifs et solutions retenues 3

2.1 Composants applicatifs et ports d'écoute 5
2.2 Comparaison des données persistantes des deux services 6

3.1 Technologies mobilisées et justification 7
3.2 Endpoints de Customer-Service 8
3.3 Endpoints de EBank-Service 13

5.1 Comportement attendu et observé du service bancaire 28
5.2 Principaux jalons de l'historique Git 29
5.3 Synthèse des vérifications réalisées 29

Liste des abréviations

Acronyme	Signification
API	Application Programming Interface – interface de programmation applicative
BDCC	Big Data and Cloud Computing
BOM	Bill of Materials – gestion cohérente des versions Maven
CRUD	Create, Read, Update, Delete
DTO	Data Transfer Object – objet de transfert de données
HTTP	Hypertext Transfer Protocol
JPA	Jakarta Persistence API
JSON	JavaScript Object Notation
JVM	Java Virtual Machine
REST	Representational State Transfer
UUID	Universally Unique Identifier

Introduction générale

Les architectures distribuées occupent une place importante dans la conception des systèmes d'information contemporains. La décomposition d'une application en services spécialisés favorise la modularité, mais elle transforme également des appels locaux en échanges réseau et impose la résolution de nouvelles questions : localisation des services, accès centralisé, propriété des données, propagation des défaillances et supervision.

Ce projet vise à étudier ces mécanismes à travers un cas d'école bancaire. Le choix de deux domaines fonctionnels – les clients et les comptes – permet d'aborder des opérations métier simples, tout en mettant en évidence une dépendance interservices : l'affichage d'un compte peut requérir la consultation des données d'un client hébergées ailleurs. Le système est développé dans un environnement local afin de privilégier l'expérimentation et la compréhension des composants.

Le rapport s'organise en cinq chapitres. Le premier présente le contexte et les objectifs. Le deuxième expose la conception distribuée et les modèles de données. Le troisième décrit la réalisation des services métier et leur validation. Le quatrième étudie Eureka et la passerelle. Le cinquième explique la communication OpenFeign, la résilience et le bilan des expérimentations.

Chapitre I

Contexte et fondements du projet

1.1 Introduction

Ce premier chapitre situe le problème technique et précise les apprentissages visés. La solution n'est pas destinée à un usage bancaire en production : elle sert de support expérimental à la compréhension de l'écosystème Spring appliqué aux microservices.

1.2 Problématique et motivation

Dans une application monolithique, les fonctions de gestion des clients et des comptes peuvent partager les mêmes modèles, transactions et accès aux données. Une architecture répartie exige au contraire de définir explicitement ce qui appartient à chaque service et d'organiser les échanges entre eux. Le principal enjeu devient la conservation d'une interface cohérente malgré la distribution des responsabilités.

La problématique étudiée est donc la suivante : *comment concevoir des services métier autonomes, accessibles par une passerelle commune, capables de se découvrir mutuellement et de fournir un résultat utile même lorsqu'une dépendance distante devient indisponible ?*

1.3 Objectifs fonctionnels et techniques

Les objectifs fonctionnels portent sur la création et la consultation de clients, la création et la consultation de comptes, le filtrage des comptes par client et l'affichage enrichi des comptes. Les objectifs techniques comprennent la persistance distincte, l'exposition REST, la découverte dynamique, le routage par chemin HTTP, la communication interservices et la mise en œuvre d'un repli.

Table 1.1 – Correspondance entre objectifs et solutions retenues

Objectif	Solution technique
Gestion indépendante des clients et des comptes	Deux applications Spring Boot et deux bases H2 distinctes.
Exposition d'une API	Contrôleurs REST utilisant GET et POST.
Découverte des instances	Serveur Netflix Eureka sur le port 8761.
Point d'entrée unifié	Spring Cloud Gateway sur le port 9999.
Communication distante	Interface déclarative OpenFeign et résolution par Eureka.
Continuité en cas de panne	Spring Cloud Circuit Breaker / Resilience4j et fallback.
Traçabilité	Compilations, scénarios Postman, captures et commits Git.

1.4 Microservices et indépendance des données

Un microservice est une unité logicielle spécialisée et exécutable indépendamment. Dans cette réalisation, `customer-service` détient l'information de référence des clients tandis que `ebank-service` détient les comptes bancaires. Un `customerId` enregistré dans un compte constitue une référence logique, non une clé étrangère vers une table distante. Cette séparation illustre la maîtrise locale des données, tout en rendant nécessaire un appel HTTP lorsque des attributs d'un client doivent être affichés avec ceux d'un compte.

1.5 Méthodologie de développement

La réalisation a adopté un déroulement incrémental : initialisation Maven et Spring Boot, définition des entités JPA, création des repositories, implémentation des règles métier, publication des contrôleurs HTTP, puis ajout des composants distribués. À chaque jalon, des compilations et des tests fonctionnels ont permis de corriger les erreurs avant de poursuivre. Les fonctionnalités ont été historisées par des commits Git distincts.

1.6 Conclusion

Le contexte et les objectifs établissent la nécessité de combiner les fondements classiques d'une API REST avec une infrastructure de découverte et de résilience. Le chapitre suivant présente cette architecture sous une forme organisée.

Chapitre II

Conception du système distribué

2.1 Vue d'ensemble

La solution repose sur quatre applications Spring Boot indépendantes. Les deux services métier possèdent des bases H2 autonomes. Le registre Eureka connaît les instances disponibles. La passerelle utilise les informations de découverte pour acheminer les appels clients ; OpenFeign permet, pour sa part, à EBank-Service de solliciter Customer-Service. La figure 4.4 montre les applications effectivement visibles dans le tableau de bord après intégration.

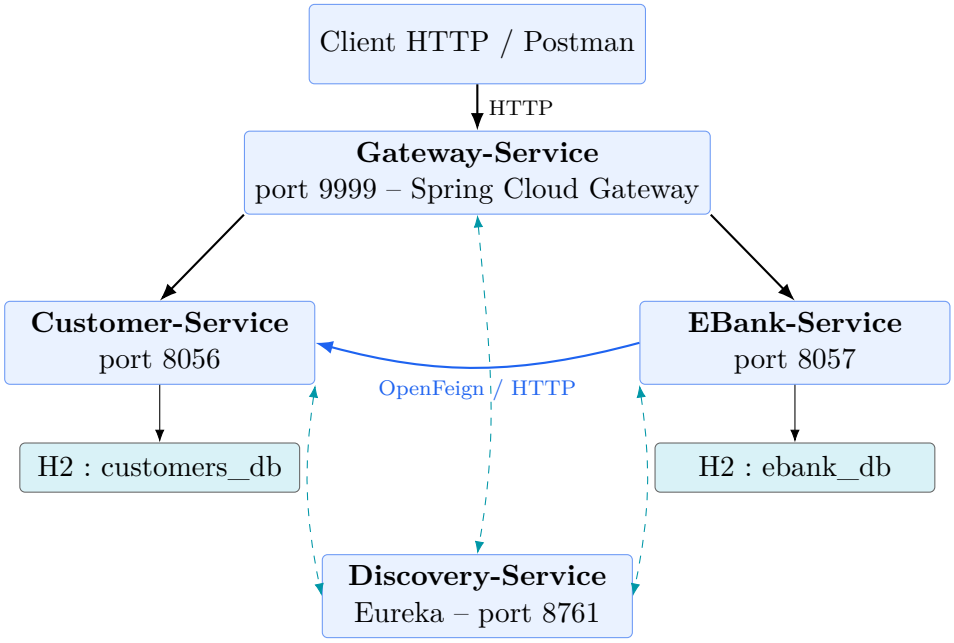


Figure 2.1 – Architecture fonctionnelle de la partie I : services métier, découverte, passerelle et données isolées

2.2 Rôle et adressage des composants

Table 2.1 – Composants applicatifs et ports d'écoute

Composant	Port	Responsabilité
Customer-Service	8056	Gestion persistante des entités client et opérations REST.
EBank-Service	8057	Gestion des comptes et enrichissement distant.
Discovery-Service	8761	Annuaire des instances Eureka.
Gateway-Service	9999	Aiguillage HTTP central et résolution `lb://`.

2.3 Architecture en couches des services métier

Chaque service métier sépare le contrôleur web, le service applicatif, le repository JPA et l'entité persistante. Le contrôleur transforme les requêtes HTTP en appels métier ; le service applique les contraintes et orchestre les traitements ; le repository réalise l'accès aux données via Spring Data JPA. Cette répartition facilite l'évolution des règles sans modifier systématiquement les routes REST.

2.4 Modèle de données

2.4.1 Client

L'entité `Customer` comprend un identifiant de type `Long`, un nom et une adresse électronique. L'identifiant est généré à l'aide de `GenerationType.IDENTITY`. Les annotations Lombok génèrent les accesseurs, constructeurs et un constructeur de type *builder*.

2.4.2 Compte bancaire

L'entité `BankAccount` comprend un identifiant textuel généré sous forme d'UUID, une date de création, un solde réel `Double`, un type de compte et un `customerId`. L'énumération de type de compte comprend `CURRENT_ACCOUNT` et `SAVING_ACCOUNT`. Le champ additionnel `customer`, marqué `@Transient`, n'est pas stocké dans H2 : il est ajouté lors de la composition de la réponse HTTP.

Table 2.2 – Comparaison des données persistantes des deux services

Critère	Customer	BankAccount
Clé primaire	`Long` généré	`String` (UUID)
Principaux attributs	`name`, `email`	`balance`, `type`, `createdAt`, `customerId`
Base de données	`customers_db`	`ebank_db`
Lien distant	Aucun	Identifiant logique `customerId`, sans jointure JPA distante
Donnée transitoire	Aucune	`Customer customer` pour la réponse

2.5 Conception des échanges HTTP

L'API de chaque ressource suit les conventions essentielles de REST : `GET` pour la lecture, `POST` pour la création, `201 Created` et l'en-tête `Location` après insertion, `404 Not Found` lorsqu'une ressource individuelle locale n'existe pas, et `400 Bad Request` lorsque les règles métier déclenchent une exception de validation.

2.6 Limites du modèle de démonstration

Les bases H2 utilisées sont en mémoire : leur contenu est perdu à l'arrêt d'une application. Le modèle ne met pas en œuvre de transaction distribuée et ne garantit pas l'existence d'un client au seul motif qu'un compte contient son identifiant. Ces propriétés doivent être comprises comme des simplifications pédagogiques, et non comme des caractéristiques souhaitables d'une plateforme bancaire réelle.

2.7 Conclusion

L'architecture distingue clairement les données possédées localement, les échanges réseau et l'infrastructure transverse. La réalisation est détaillée dans les chapitres suivants.

Chapitre III

Réalisation des services métier

3.1 Environnement de développement

Le projet a été construit sous macOS avec Visual Studio Code, Java 21, Maven et le générateur Spring Initializr. Les quatre modules utilisent Spring Boot 3.5.10 ainsi que le BOM Spring Cloud 2025.0.3. Les dépendances métier essentielles sont Spring Web, Spring Data JPA, H2, Actuator, Lombok et Spring Boot Test. La commande `./mvnw clean compile` permet de vérifier la compilation indépendamment pour chaque projet.

Table 3.1 – Technologies mobilisées et justification

Technologie	Contribution au projet
Java 21	Langage d'implémentation et runtime JVM.
Spring Boot 3.5.10	Initialisation, injection de dépendances, gestion de configuration.
Spring Data JPA	Repositories et mapping objet-relationnel.
H2	Persistance transitoire pour les essais locaux.
Spring Cloud 2025.0.3	Alignement des dépendances de découverte, passerelle et résilience.
Lombok	Réduction du code répétitif des objets métier.
Postman, Git, GitHub	Tests HTTP et traçabilité incrémentale.

3.2 Implémentation de Customer-Service

3.2.1 Initialisation et persistance

Le service client est exposé sur le port `8056` avec une base H2 nommée `customers_db`. L'annotation `@Entity` définit la correspondance relationnelle, tandis que le repository

`CustomerRepository`, dérivé de `JpaRepository` met à disposition les opérations CRUD courantes sans implémentation manuelle du stockage.

Listing 3.1 – Paramètres principaux de Customer-Service

```
spring.application.name=customer-service
server.port=8056
spring.datasource.url=jdbc:h2:mem:customers_db
spring.h2.console.enabled=true
spring.jpa.hibernate.ddl-auto=create-drop
spring.jpa.show-sql=true
```

3.2.2 Service métier et contrôleur HTTP

La classe `CustomerService` propose la consultation de tous les clients, la recherche par identifiant et la création d'un client. Deux validations simples empêchent l'enregistrement d'un nom ou d'une adresse électronique vide. Le contrôleur `CustomerRestController`, accessible sous `/customers`, délègue au service métier et transforme les résultats en codes HTTP adaptés.

Table 3.2 – Endpoints de Customer-Service

Méthode	Chemin	Fonction
GET	`/hello`	Vérification élémentaire du service.
GET	`/customers`	Liste des clients.
GET	`/customers/{id}`	Détail d'un client ; 404 si absent.
POST	`/customers`	Création ; 201 ou 400 suivant la validation.

3.2.3 Premières validations

La vérification s'est déroulée en plusieurs étapes : démarrage du processus Spring Boot, interrogation de l'endpoint de contrôle, compilation, tests automatisés du repository et du service métier, puis consultation de la console H2 et des réponses HTTP. Les figures suivantes conservent des traces de ces jalons.

3.2.4 Tests automatisés du service client

Le dépôt contient notamment `CustomerRepositoryTest`, annoté `@DataJpaTest`, qui enregistre puis recherche un client. `CustomerServiceTest`, annoté `@SpringBootTest` et `@Transactional`, vérifie la création et le rejet d'un nom manquant. Les sorties Maven et les essais Postman documentent une validation progressive ; elles ne constituent pas une mesure exhaustive de couverture de tests.

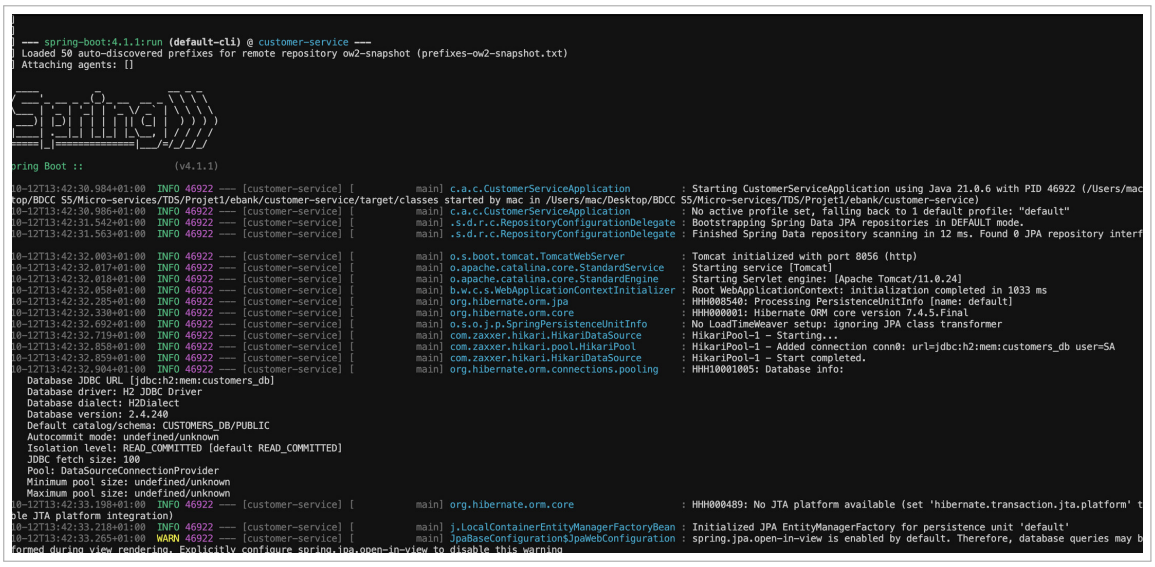


Figure 3.1 – Démarrage initial du service client et traces Spring Boot

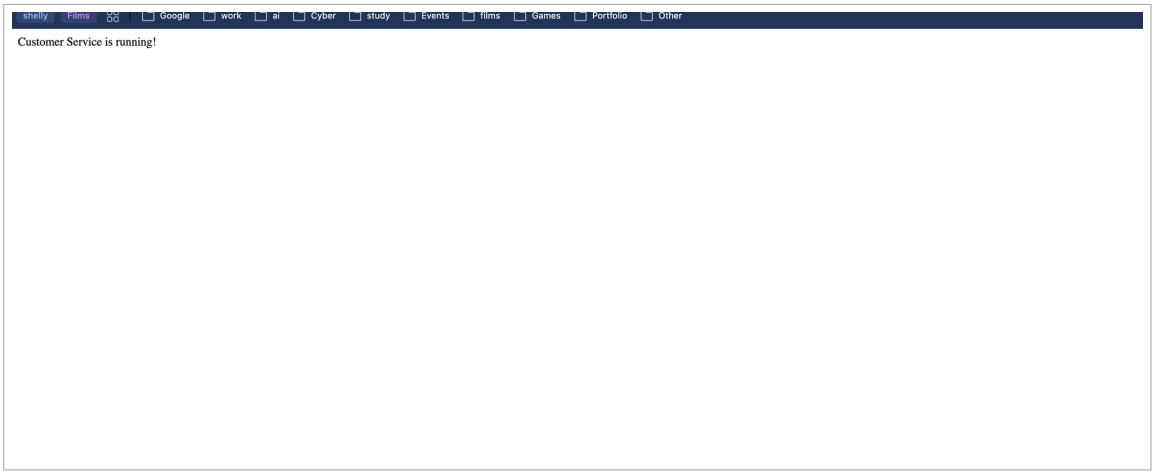


Figure 3.2 – Vérification HTTP du point d'entrée de contrôle /hello

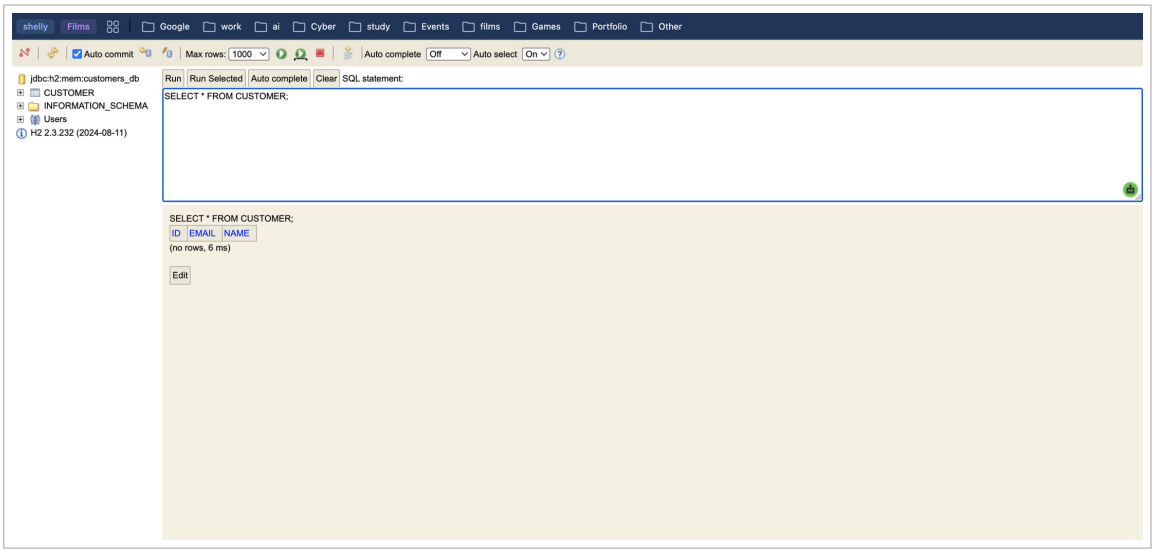


Figure 3.3 – Exploration de la base H2 des clients à la création

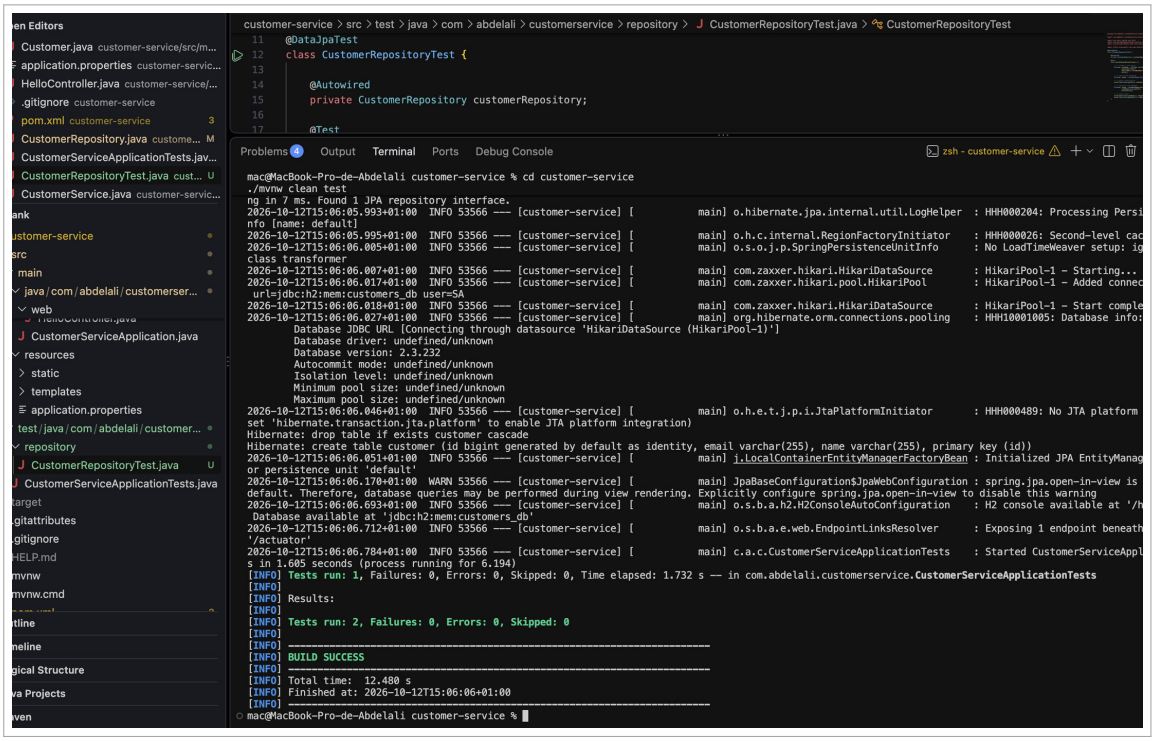


Figure 3.4 – Compilation et vérifications du service client dans VS Code

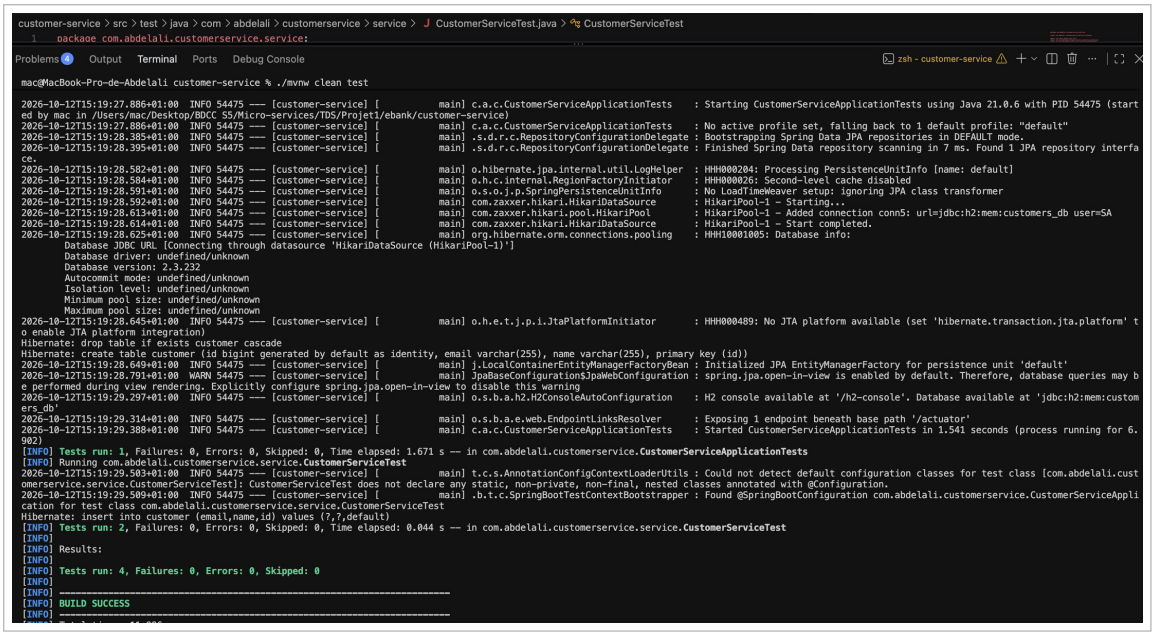


Figure 3.5 – Exécution des tests du composant métier client

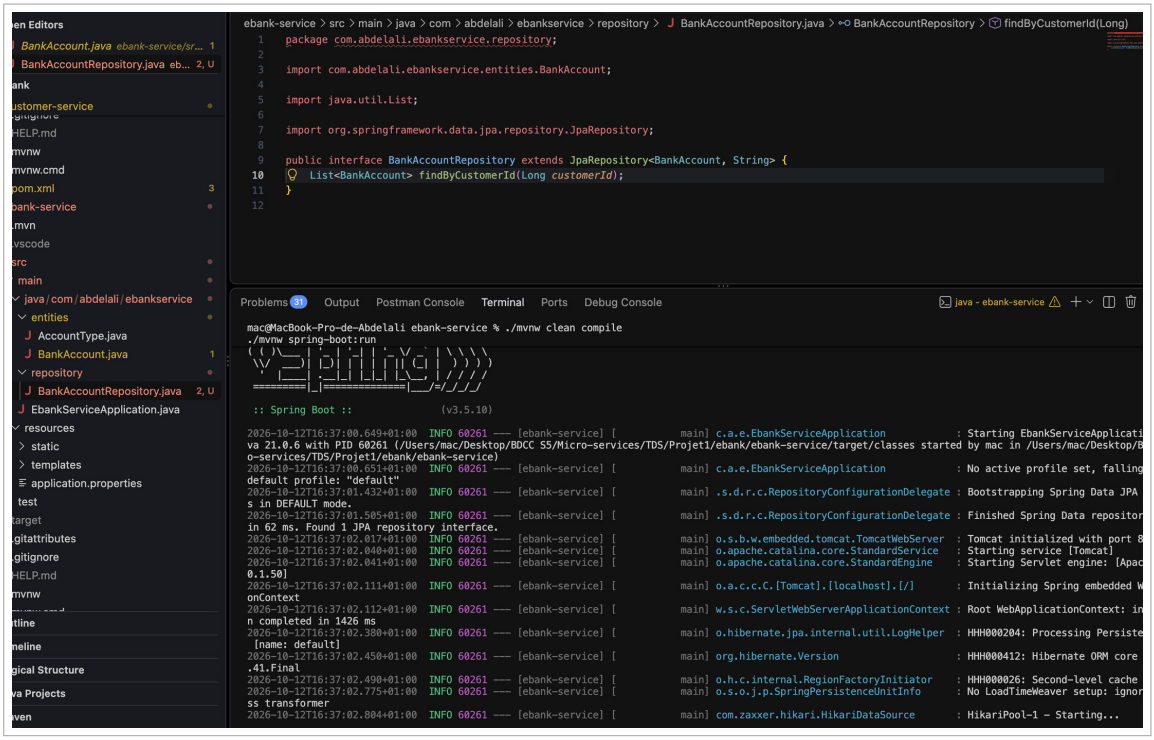


Figure 3.6 – Implémentation et lancement des composants du service client

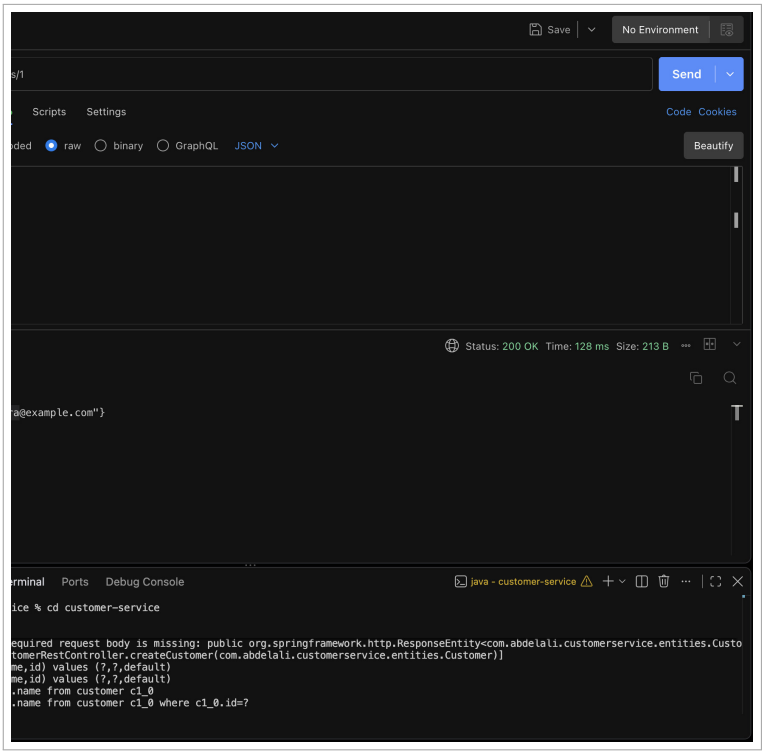


Figure 3.7 – Création d'un client par requête HTTP POST

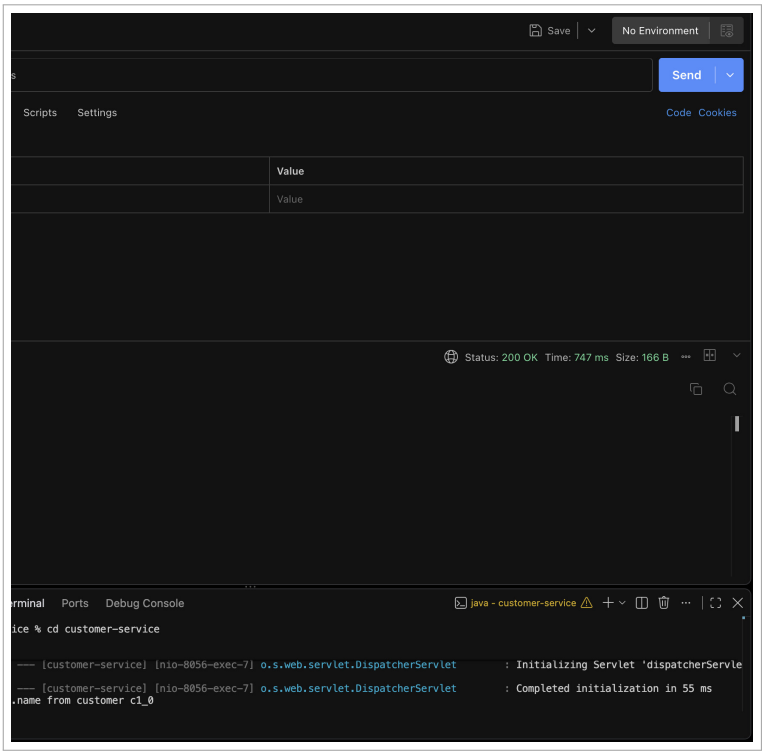


Figure 3.8 – Consultation des clients par requête HTTP GET directe

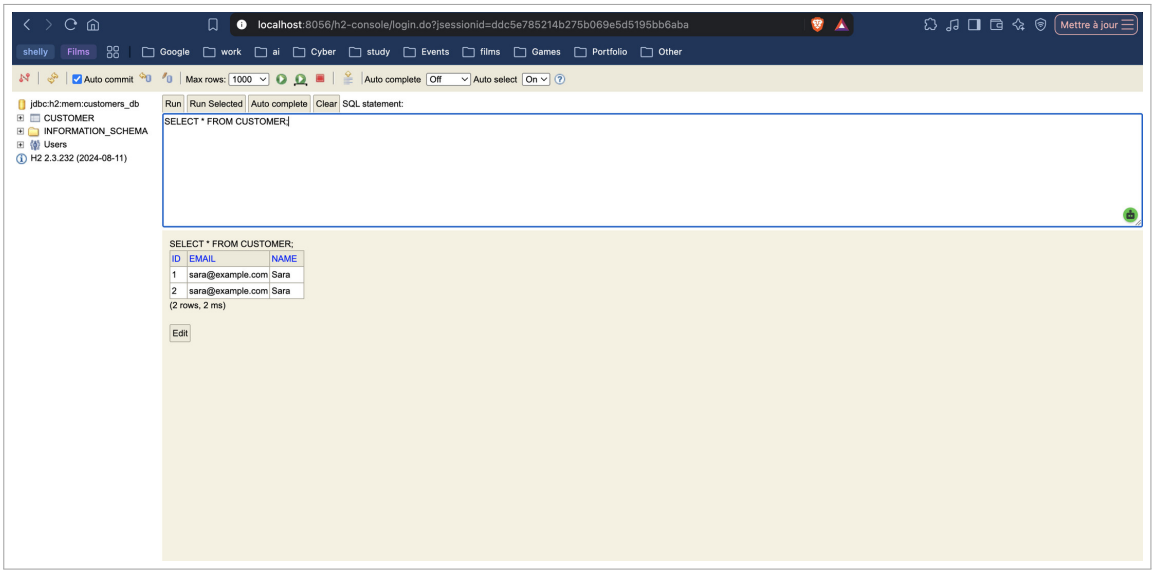


Figure 3.9 – Inspection des enregistrements de la table CUSTOMER

3.3 Implémentation de EBank-Service

3.3.1 Entité et repository

Le second service s'exécute sur le port `8057` et utilise `jdbc:h2:mem:ebank_db`. L'entité `BankAccount` associe les champs décrits précédemment à un repository dérivé de `JpaRepository`. La méthode dérivée `findByCustomerId(Long)` permet de sélectionner les comptes liés à un même identifiant client.

3.3.2 Règles métier

La couche de service impose un solde présent, fini et positif ou nul ; elle requiert également un identifiant client et un type de compte. Lors de la création, un UUID et une date de création sont définis avant l'enregistrement. Ces contrôles améliorent la cohérence locale du modèle, sans représenter une réglementation bancaire complète.

3.3.3 Publication de l'API

Le contrôleur `BankAccountRestController` expose des ressources de lecture et de création. Sa responsabilité consiste à retourner les entités ou des réponses appropriées lorsqu'un identifiant manque ou qu'une entrée est invalide.

Table 3.3 – Endpoints de EBank-Service

Méthode	Chemin	Fonction
GET	`/accounts`	Liste des comptes.
GET	`/accounts/{id}`	Consultation d'un compte individuel.
GET	`/accounts/customer/{customerId}`	Liste des comptes d'un client.
POST	`/accounts`	Création d'un compte et retour 201.

3.3.4 Validation en environnement local

Les captures illustrent le code du service bancaire, son démarrage, l'inspection de la structure des données et des réponses Postman correspondant aux opérations de création et de consultation. L'usage d'H2 facilite la répétition des essais, à condition de recréer les données après chaque arrêt du processus.

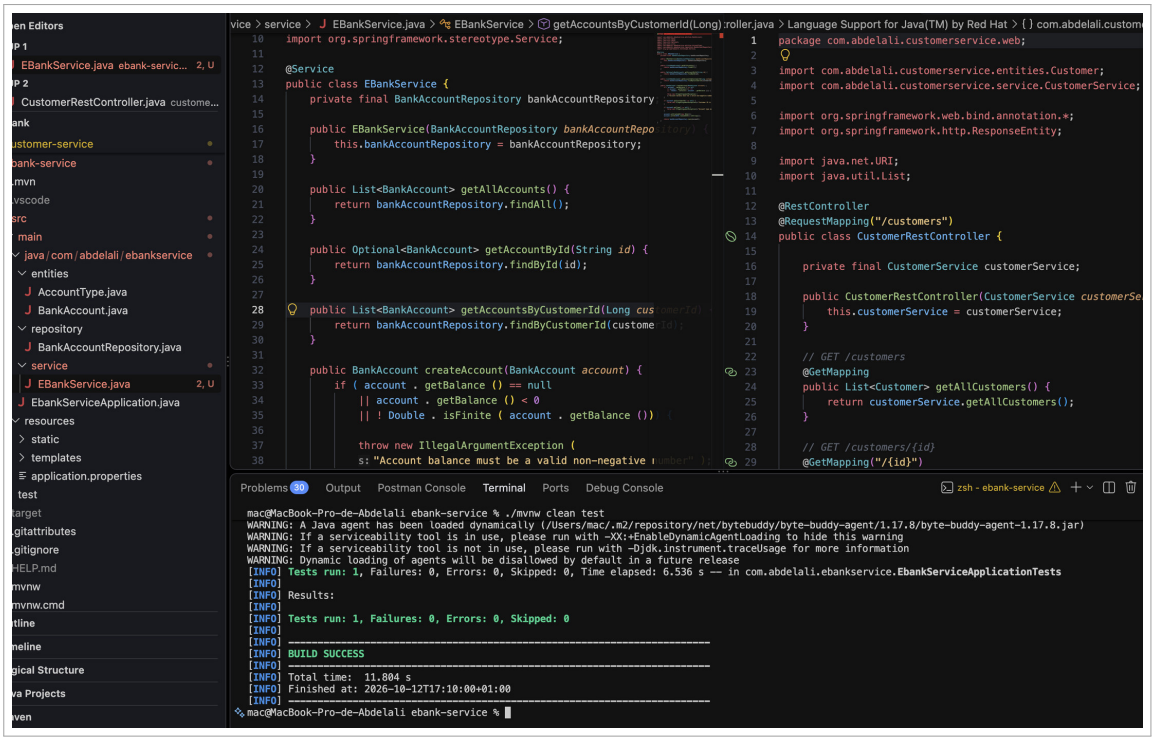


Figure 3.10 – Code du contrôleur bancaire et vérification de son exécution

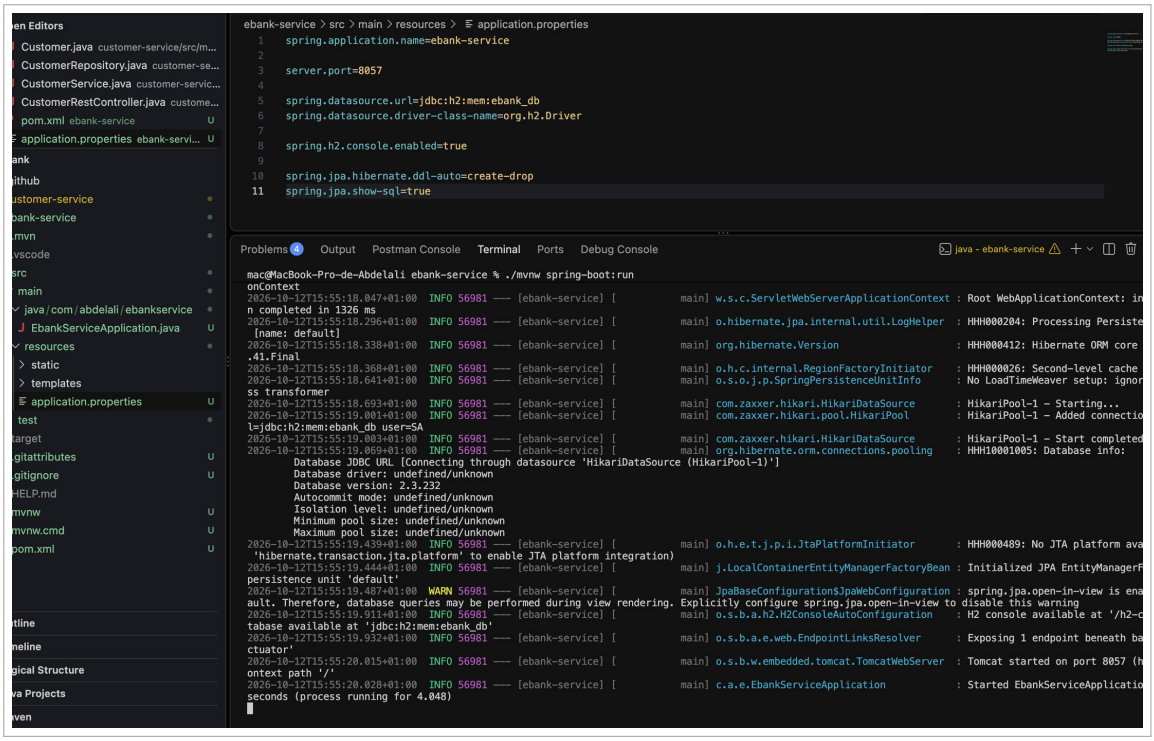


Figure 3.11 – Compilation et exécution du service bancaire

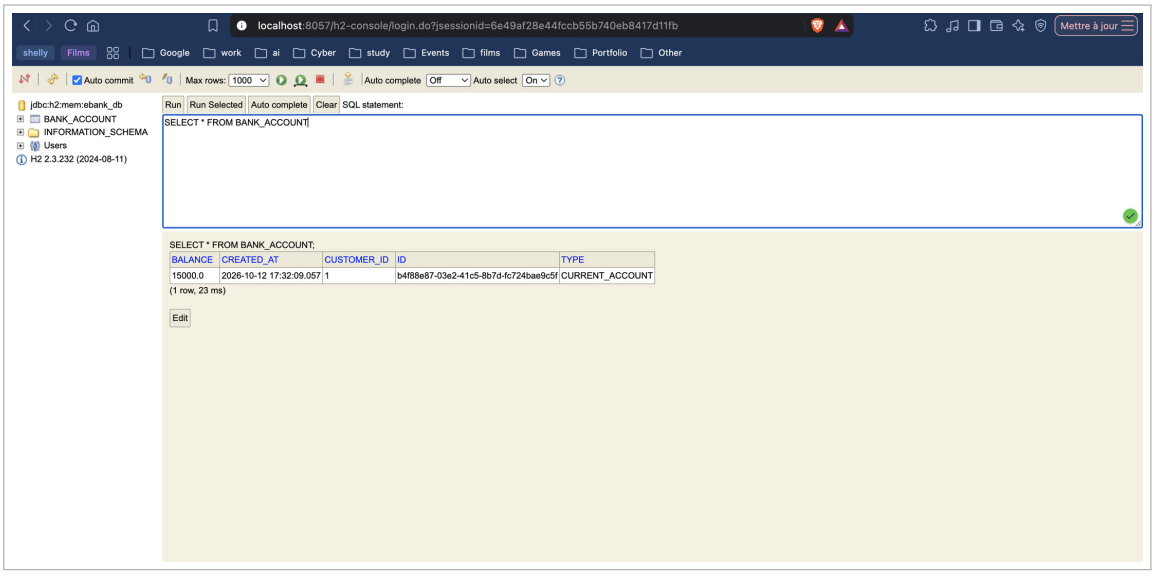


Figure 3.12 – Inspection initiale de la structure H2 bancaire

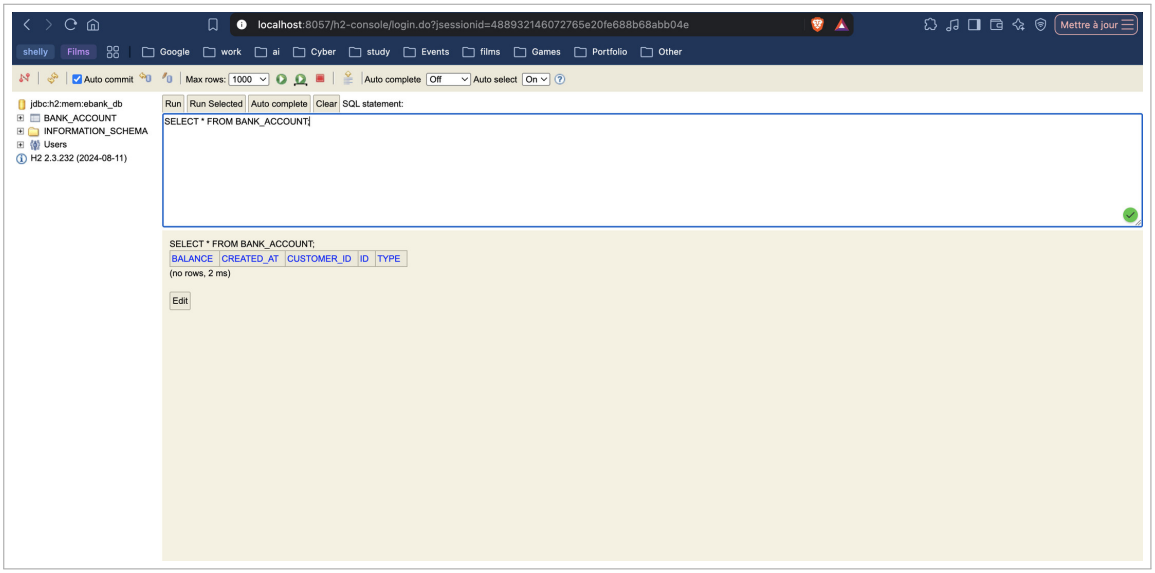


Figure 3.13 – Lecture de la table BANK_ACCOUNT dans H2

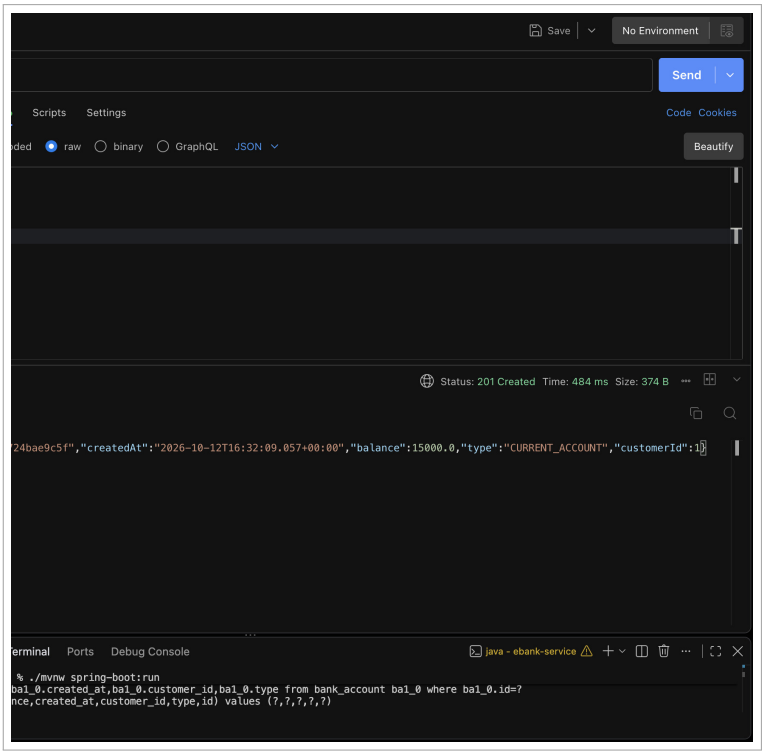


Figure 3.14 – Création d'un compte bancaire par requête POST directe

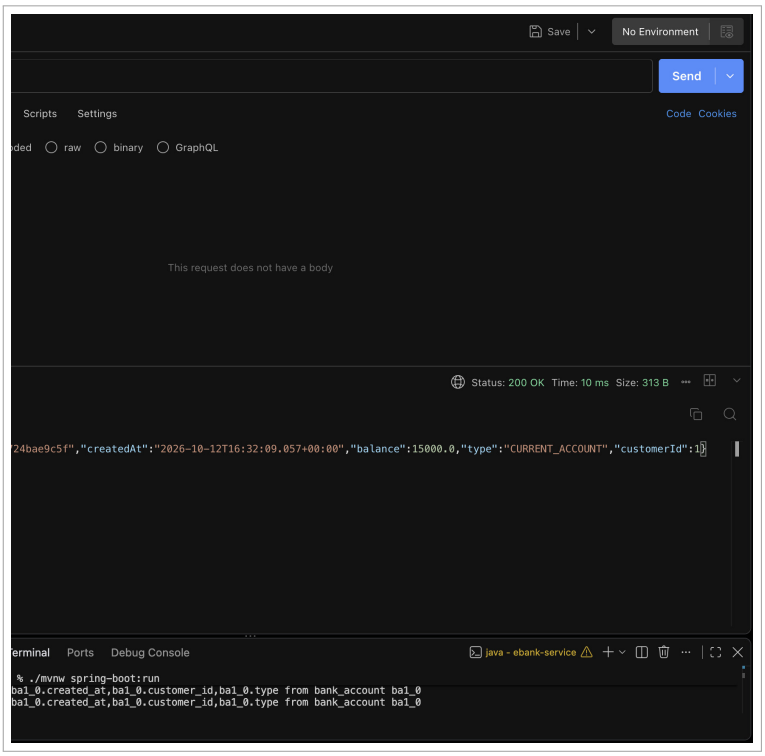


Figure 3.15 – Consultation d'un compte bancaire par requête GET directe

3.4 Conclusion

Les deux services métier disposent désormais de leurs propres entités, repositories, traitements et contrôleurs REST. Cette base fonctionnelle permet l'introduction de la découverte dynamique et de la passerelle dans le chapitre suivant.

Chapitre IV

Découverte de services et passerelle HTTP

4.1 Netflix Eureka : registre de découverte

Le serveur de découverte, nommé `discovery-service`, est exécuté sur le port `8761`. L'annotation `@EnableEurekaServer` active la fonction de registre. Le mode autonome est obtenu en désactivant l'auto-enregistrement du serveur et la récupération d'un registre externe.

Listing 4.1 – Paramètres Eureka du serveur de découverte

```
spring.application.name=discovery-service
server.port=8761
eureka.client.register-with-eureka=false
eureka.client.fetch-registry=false
```

La console Eureka affiche d'abord un registre vide. Dès que les clients Eureka sont installés et que les services métier démarrent, ces derniers publient leur identité et renouvellent périodiquement leur inscription. Les figures documentent successivement l'état initial, puis l'arrivée du service client et du service bancaire.

4.2 Enregistrement des applications métier

Les modules Customer-Service et EBank-Service utilisent la dépendance `client Eureka de Spring Cloud` et renseignent l'adresse de découverte suivante :

```
eureka.client.service-url.defaultZone=http://localhost:8761/eureka/
```

Cette approche évite de maintenir manuellement une liste d'hôtes auprès des autres composants. Le registre fournit des instances associées à leur nom logique et à leur adresse

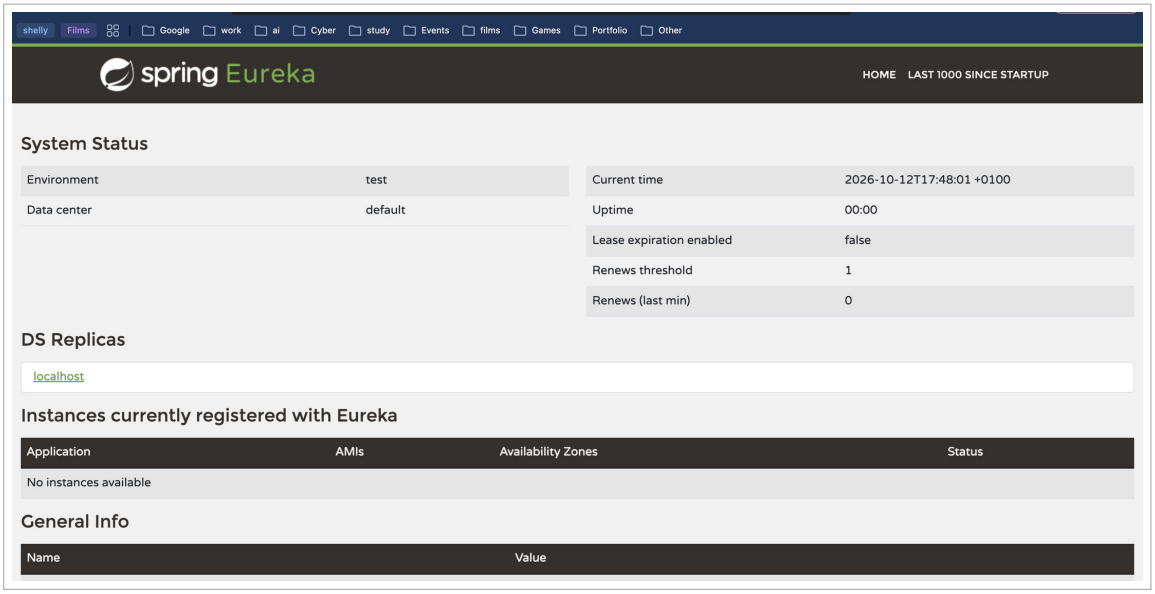


Figure 4.1 – Console Eureka au démarrage, sans instance inscrite

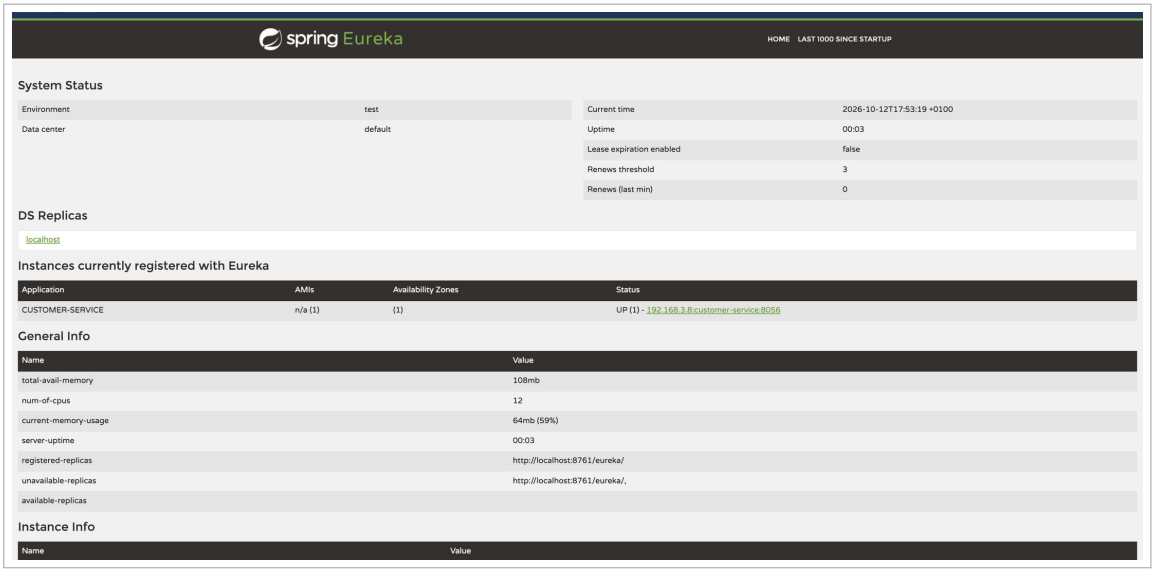


Figure 4.2 – Enregistrement de CUSTOMER-SERVICE dans Eureka

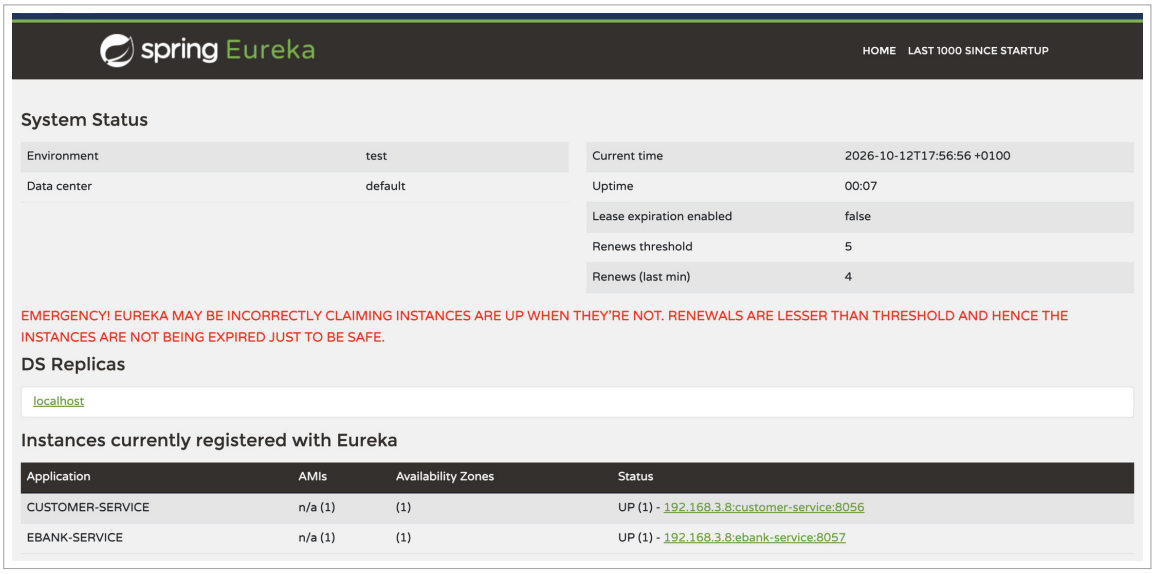


Figure 4.3 – Enregistrement simultané de CUSTOMER-SERVICE et EBANK-SERVICE

réseau.

4.3 Spring Cloud Gateway

La passerelle expose le port `9999` et s'appuie sur le serveur Gateway WebFlux. Deux routes utilisent des prédicats de chemin pour transférer les requêtes vers les services cibles. Le schéma `lb://` sollicite Spring Cloud LoadBalancer afin de sélectionner une instance enregistrée. Le service de découverte et le mécanisme de répartition restent distincts du traitement des données métier.

Listing 4.2 – Règles HTTP configurées dans Gateway-Service

```
spring.application.name=gateway-service
server.port=9999
spring.cloud.gateway.server.webflux.routes[0].id=customer-service-route
spring.cloud.gateway.server.webflux.routes[0].uri=lb://customer-service
spring.cloud.gateway.server.webflux.routes[0].predicates[0]=Path=/customers,/customers/**
spring.cloud.gateway.server.webflux.routes[1].id=ebank-service-route
spring.cloud.gateway.server.webflux.routes[1].uri=lb://ebank-service
spring.cloud.gateway.server.webflux.routes[1].predicates[0]=Path=/accounts,/accounts/**
```

4.4 Routage de couche applicative et répartition

La passerelle examine le chemin d'une requête HTTP pour choisir le service destinataire. Il s'agit ainsi d'un **routage de couche 7** (couche application), par opposition à une décision de couche 4 fondée principalement sur une adresse IP et un port transport. Le LoadBalancer prend ensuite en charge la sélection d'une instance du service cible. Dans

l'expérimentation décrite, une seule instance de chaque service métier a été observée ; aucun essai quantitatif de distribution multi-instance n'est revendiqué.

4.5 Validation de la passerelle

Les essais ont permis d'atteindre les routes `/customers` et `/accounts` via le port unique `9999`. Des requêtes de création et de consultation ont été testées au travers de ce point d'entrée. L'enregistrement du module Gateway dans Eureka est présenté par la figure 4.4.

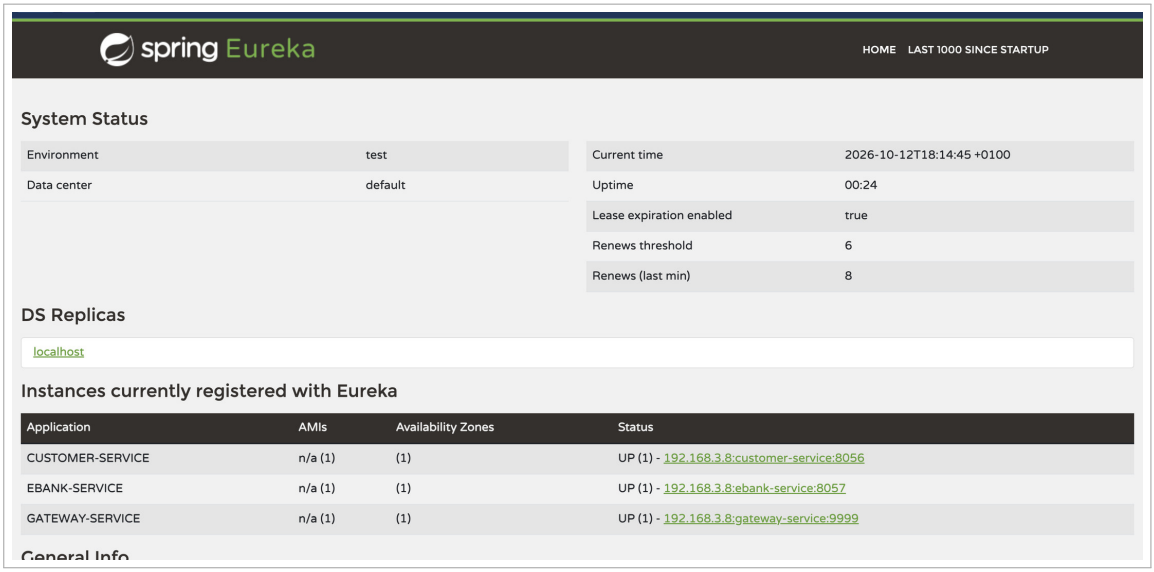


Figure 4.4 – Enregistrement des trois composants applicatifs, incluant Gateway

4.6 Précision sur les avertissements Eureka

Pendant les expérimentations, le tableau de bord a parfois affiché un avertissement relatif au *self-preservation mode* : lorsque le nombre de renouvellements est inférieur à un seuil, Eureka peut conserver temporairement certaines inscriptions afin d'éviter leur expiration abusive. Ce message ne constitue pas, à lui seul, une preuve de panne du système métier. La disponibilité effective doit aussi être vérifiée par des appels HTTP indépendants.

4.7 Conclusion

Grâce à Eureka et à Gateway, le client HTTP peut joindre les APIs métier à partir d'un point d'entrée commun. Le registre simplifie la localisation des services, tandis que

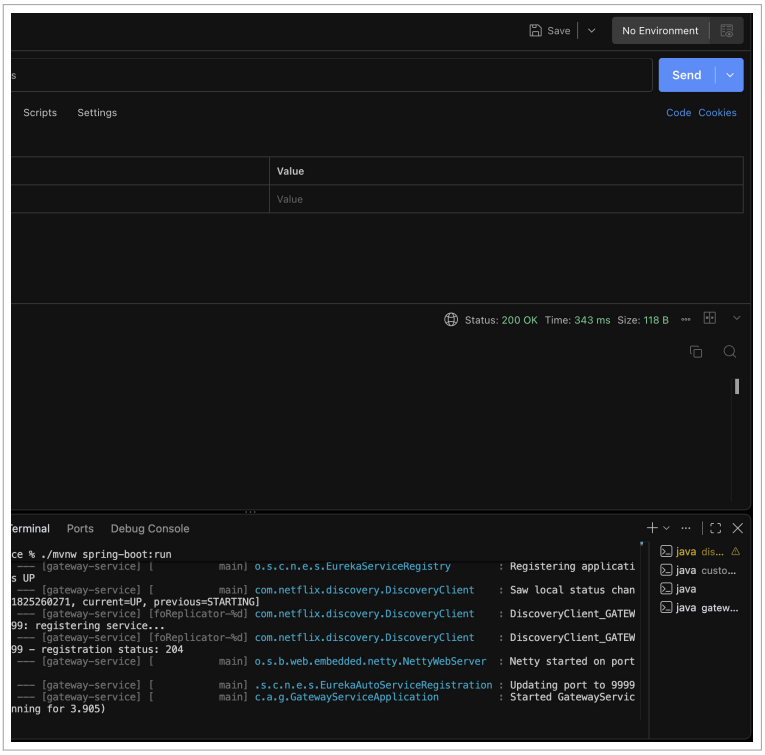


Figure 4.5 – Consultation de /customers à travers le port 9999

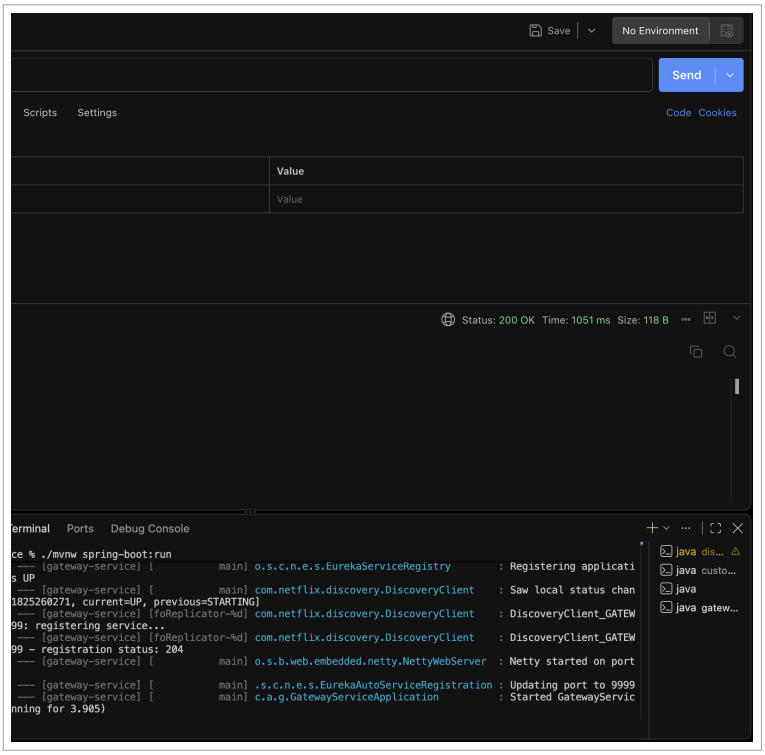


Figure 4.6 – Consultation de /accounts à travers le port 9999

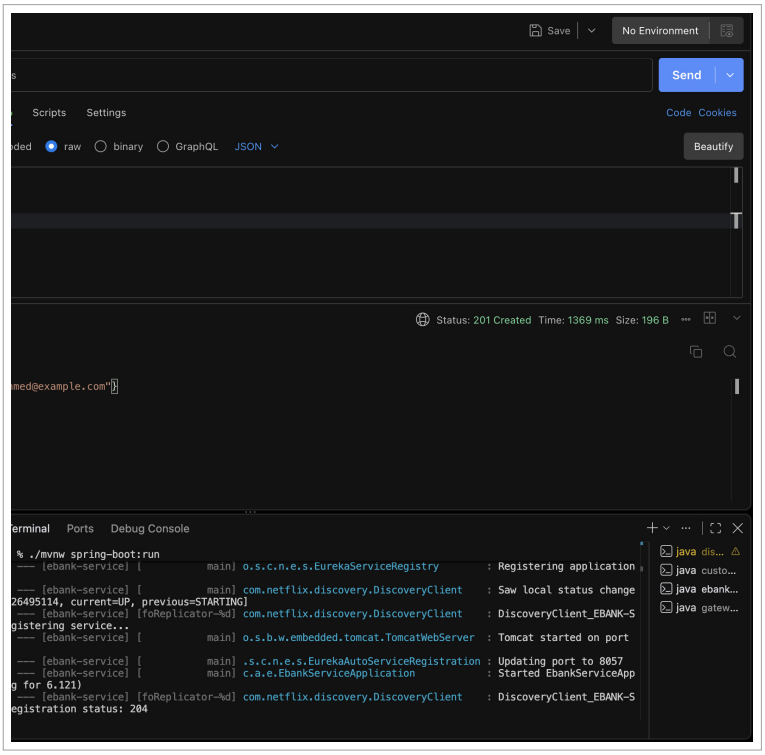


Figure 4.7 – Création d'un client à travers la passerelle HTTP

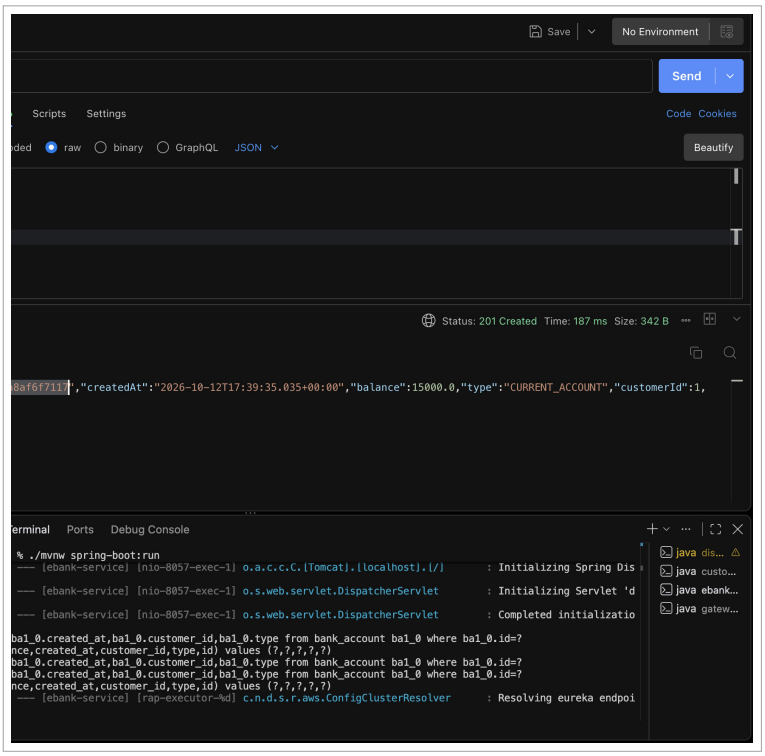


Figure 4.8 – Création d'un compte bancaire à travers la passerelle HTTP

les règles de la passerelle centralisent le routage.

Chapitre V

Communication interservices et tolérance aux pannes

5.1 Problème de composition des données

Le compte bancaire contient `customerId`, mais le nom et l'adresse électronique du client ne figurent pas dans sa base locale. Pour afficher des informations combinées sans dupliquer la table des clients, EBank-Service effectue une requête distante vers Customer-Service. Cette opération est synchrone et peut donc échouer si le service distant n'est pas disponible.

5.2 Intégration de Spring Cloud OpenFeign

La dépendance `Spring Cloud OpenFeign` est ajoutée à EBank-Service, puis `@EnableFeignClients` active la découverte des interfaces déclaratives. Le modèle `Customer` contenu dans EBank-Service est un DTO : il est distinct de l'entité persistante définie dans Customer-Service. L'interface suivante exprime directement le contrat HTTP distant.

Listing 5.1 – Contrat déclaratif Feign utilisé pour consulter un client

```
@FeignClient(name = "customer-service",
             fallback = CustomerRestClientFallback.class)
public interface CustomerRestClient {
    @GetMapping("/customers/{id}")
    Customer getCustomerById(@PathVariable("id") Long id);
}
```

5.3 Enrichissement d'une entité par un DTO distant

Dans `BankAccount`, un attribut `customer` annoté `@Transient` permet de conserver les données reçues à la sérialisation JSON, sans les persister. La méthode `getAccountById` retrouve d'abord le compte local, demande les détails du client par OpenFeign, puis affecte le DTO obtenu au compte. Si l'identifiant bancaire n'existe pas, le contrôleur conserve sa réponse `404`.

Listing 5.2 – Principe de composition d'un compte et de son client

```
return bankAccountRepository.findById(id)
    .map(account -> {
        Customer customer = customerRestClient
                .getCustomerById(account.getCustomerId());
        account.setCustomer(customer);
        return account;
    });
```

5.4 Déroulement d'une requête interservices

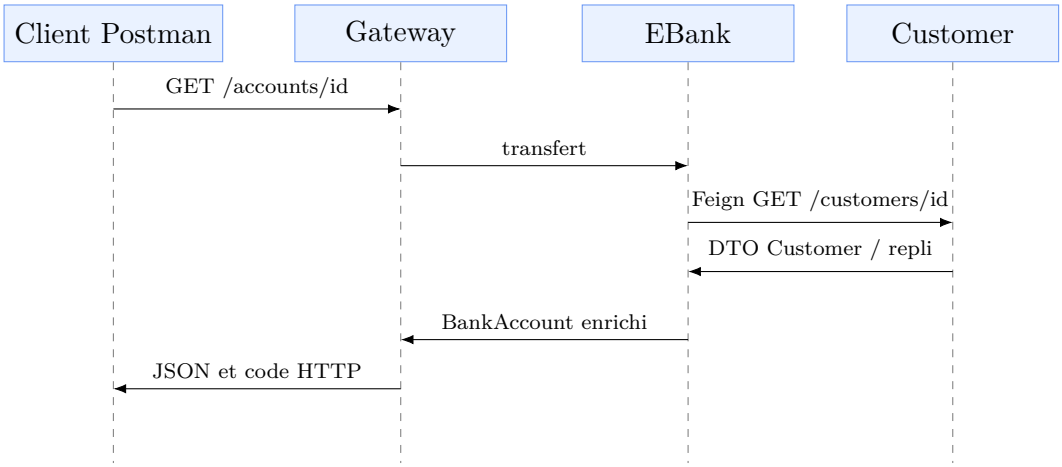


Figure 5.1 – Séquence simplifiée de récupération enrichie d'un compte bancaire

5.5 Validation fonctionnelle d'OpenFeign

Le scénario complet consiste à créer un client, puis un compte portant son identifiant, et enfin à consulter ce compte via le Gateway. Le JSON résultant comporte, outre les attributs bancaires, un objet `customer` correspondant au client demandé. La capture suivante présente ce résultat dans l'environnement de test. Les noms et valeurs numériques visibles dans les essais constituent des données de démonstration.

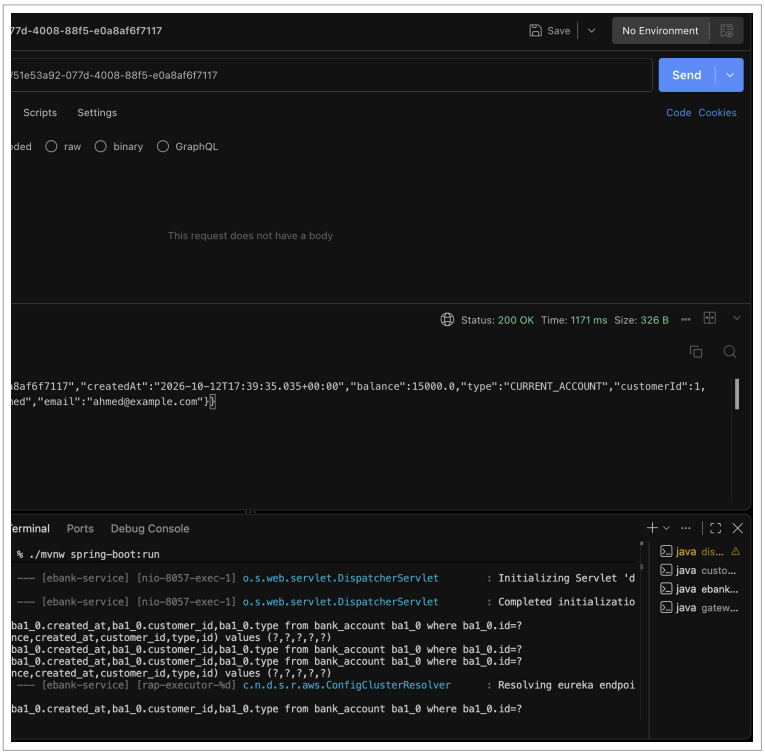


Figure 5.2 – Consultation enrichie d'un compte et de son client par OpenFeign

5.6 Circuit Breaker et mode dégradé

Un appel distant peut subir un délai, une indisponibilité ou un échec de résolution. La dépendance `Spring Cloud Circuit Breaker / Resilience4j` et la propriété suivante activent l'intégration avec les clients Feign :

```
spring.cloud.openfeign.circuitbreaker.enabled=true
```

Le circuit breaker distingue les états *CLOSED* (appels autorisés), *OPEN* (appels temporairement interrompus) et *HALF-OPEN* (essais limités pour vérifier la reprise). La stratégie de repli du projet est volontairement simple : si l'appel au service client échoue, une implémentation `CustomerRestClientFallback` retourne `null`. Le compte bancaire, qui appartient à EBank-Service, reste néanmoins disponible.

Listing 5.3 – Implémentation simplifiée du repli OpenFeign

```
@Component
public class CustomerRestClientFallback
        implements CustomerRestClient {
    @Override
    public Customer getCustomerById(Long id) {
        return null;
    }
}
```

5.7 Expérimentation de l'indisponibilité

Le test manuel a consisté à conserver EBank-Service, Eureka et Gateway actifs, à arrêter Customer-Service et à relancer la consultation d'un compte déjà enregistré. Le résultat annoncé au cours du développement confirme le maintien d'une réponse contenant les informations du compte, sans données client enrichies. **Cette expérimentation a été réalisée sans qu'une capture dédiée ait été conservée.** Elle est donc mentionnée comme validation manuelle rapportée, et non comme preuve visuelle disponible.

Table 5.1 – Comportement attendu et observé du service bancaire

Situation	Résultat attendu	État de la validation
Customer-Service disponible	Compte et DTO client dans la réponse	Requêtes HTTP et capture conservée.
Customer-Service indisponible	Compte disponible ; champ client nul	Essai manuel indiqué comme réussi ; pas de capture conservée.
Compte bancaire inexistant	Réponse 404	Comportement prévu par le contrôleur et le service local.

5.8 Limites de la stratégie de résilience

Le repli `null` démontre la continuité de service, mais masque la cause précise d'une anomalie : indisponibilité réseau, erreur HTTP ou client absent peuvent conduire à des comportements qu'il faudrait différencier. Des journaux structurés, une politique de délai et des tests automatisés de panne constitueraient des prolongements pertinents. Le travail ne comprend pas d'authentification ni de déploiement en production.

5.9 Traçabilité et progression des travaux

Le développement a été historisé dans le dépôt GitHub public du projet. Plusieurs étapes ont fait l'objet de commits ciblés, depuis l'initialisation du service client jusqu'à l'intégration du repli Resilience4j.

5.10 Bilan des validations

Le tableau ci-dessous réunit les contrôles effectivement réalisés pendant cette première partie. L'approche fournit une preuve de faisabilité locale. Elle n'est pas assimilable à une évaluation de performance, un test de charge ou un audit de sécurité.

Table 5.2 – Principaux jalons de l'historique Git

Commit	Étape fonctionnelle
`22a4959`	Initialisation du service client.
`7105abd`	Publication de l'API REST des clients.
`a6cd4a1`	Publication de l'API REST des comptes.
`9b0b3a3`	Configuration du serveur Eureka.
`0a07a4c`	Enregistrement du service client.
`8958fa4`	Enregistrement du service bancaire.
`08bf4b4`	Ajout de la passerelle HTTP.
`7428eb9`	Communication interservices avec OpenFeign.
`3a56624`	Circuit breaker et fallback.

Table 5.3 – Synthèse des vérifications réalisées

Contrôle	Éléments de résultat
Compilation Maven	Compilations réussies pendant le développement des quatre modules.
Persistance JPA/H2	Inspection des tables et opérations de création/lecture.
Tests automatisés du client	Repository et service métier : tests présents dans le dépôt.
API REST directe	Opérations GET et POST testées à l'aide de Postman.
Enregistrement Eureka	Instances métier et passerelle affichées par le registre.
Routage Gateway	Accès aux ressources depuis le port 9999.
OpenFeign	Enrichissement du compte par des informations client.
Tolérance aux pannes	Repli manuel vérifié selon le compte rendu de développement.

5.11 Conclusion

Les essais réalisés montrent la cohérence de l'architecture locale et l'intérêt d'une séparation entre services métier, découverte, routage, composition des données et résilience. La démonstration reste volontairement limitée à un contexte pédagogique.

Conclusion générale

La première partie du projet E-Bank a permis de construire une chaîne complète d'interactions entre quatre applications Spring Boot : deux services métier, un registre Eureka et une passerelle Gateway. La mise en œuvre progressive de JPA et H2 a clarifié la responsabilité des données ; les APIs REST ont servi de support aux essais fonctionnels ; l'intégration d'Eureka a permis la résolution dynamique de noms de services. Le Gateway a ensuite proposé un point d'entrée unique, tandis qu'OpenFeign a matérialisé la communication synchrone entre les domaines client et compte bancaire.

L'ajout de Resilience4j a mis en évidence un principe central des systèmes distribués : l'indisponibilité d'une dépendance ne doit pas nécessairement empêcher l'accès aux informations locales. La fonctionnalité de repli développée conserve ainsi les données du compte malgré l'absence temporaire des détails du client. La progression est documentée par les captures exploitables et par l'historique Git du projet.

Les perspectives d'évolution comprennent l'intégration de Swagger/OpenAPI, une base durable, la sécurité des accès, une validation plus poussée de la cohérence interservices, des essais multi-instances, des métriques et des tests automatisés de résilience. Ces éléments ne sont pas présentés comme réalisés dans la présente partie. L'objectif pédagogique – comprendre, implémenter et expérimenter les principales briques d'une architecture microservices Spring – est atteint pour le périmètre testé.

Bibliographie et ressources

---

1. Spring, *Spring Boot Reference Documentation*, https://docs.spring.io/spring-boot/.
2. Spring, *Spring Cloud Netflix / Eureka*, https://docs.spring.io/spring-cloud-netflix/reference/.
3. Spring, *Spring Cloud Gateway Server WebFlux*, https://docs.spring.io/spring-cloud-gateway/reference/.
4. Spring, *Spring Cloud OpenFeign*, https://docs.spring.io/spring-cloud-openfeign/reference/.
5. Spring, *Spring Cloud Circuit Breaker*, https://docs.spring.io/spring-cloud-circuitbreaker/reference/.
6. Spring, *Spring Data JPA*, https://docs.spring.io/spring-data/jpa/reference/.
7. Resilience4j, *Documentation*, https://resilience4j.readme.io/.
8. Apache Maven, *Maven Documentation*, https://maven.apache.org/guides/.
9. Git, *Documentation officielle*, https://git-scm.com/doc.
10. Dépôt de réalisation, https://github.com/AbdelaliSaadali/bdcc-s5-ebank-microservices, branche `main`, version examinée : `3a56624` (12 octobre 2026).

Annexe A

Éléments complémentaires

A.1 Structure synthétique des projets Maven

```
ebank/
  customer-service/
    pom.xml
    src/main/java/com/abdelali/customerservice/
       entities/ repository/ service/ web/
  ebank-service/
    pom.xml
    src/main/java/com/abdelali/ebankservice/
       entities/ repository/ service/ web/
       clients/ model/
  discovery-service/
    pom.xml
    src/main/java/com/abdelali/discoveryservice/
  gateway-service/
    pom.xml
    src/main/java/com/abdelali/gatewayservice/
```

A.2 Exemples de requêtes pour la reproduction

Les charges utiles ci-dessous sont des exemples illustratifs ; les identifiants réels sont attribués au moment des requêtes.

Listing A.1 – Création d'un client via Gateway

```
POST http://localhost:9999/customers
Content-Type: application/json

{"name":"Ahmed", "email":"ahmed@example.com"}
```

Listing A.2 – Création d'un compte puis consultation enrichie

```
POST http://localhost:9999/accounts
Content-Type: application/json

{"balance":15000.0,"type":"CURRENT_ACCOUNT","customerId":1}

GET http://localhost:9999/accounts/{uuid}
```

A.3 Ordre de lancement recommandé

Il est recommandé de démarrer d'abord le serveur de découverte, puis les services métier et enfin la passerelle, chacun dans son propre terminal. L'ordre est utile au diagnostic des enregistrements initiaux ; les clients Eureka peuvent néanmoins tenter de se reconnecter ultérieurement si le registre n'est pas immédiatement disponible.

```
cd discovery-service && ./mvnw spring-boot:run
cd customer-service && ./mvnw spring-boot:run
cd ebank-service && ./mvnw spring-boot:run
cd gateway-service && ./mvnw spring-boot:run
```

Les commandes sont à exécuter dans des terminaux distincts à partir de la racine du dépôt.

A.4 Note de gestion des preuves visuelles

L'archive initiale contient 25 images. Les 24 images directement associées aux réalisations du présent projet ont été sélectionnées, recadrées pour améliorer leur lisibilité et référencées en début de rapport. La capture extérieure à la solution concernant la gestion d'une clé API n'a pas été intégrée afin de protéger les informations personnelles et de garantir la pertinence du livrable. Aucun cliché du scénario de repli Resilience4j n'a été ajouté, conformément au matériel réellement conservé.